

PRÁCTICA: MEDICIÓN DE RENDIMIENTO PARALLEL

Estimación de π mediante Algoritmo de Monte Carlo con POSIX Threads

ESTUDIANTE	MUESTRA TOTAL (N)	MECANISMO SYNC	CONFIGURACIÓN CPU
[Nombre del Alumno]	100,000,000 Puntos	Mutex / Acum. Local	8 Hilos Virtuales

1. Definición de Métricas Evaluadas

Aceleración (Speedup) $S(p) = T(1) / T(p)$	Eficiencia Paralela $E(p) = S(p) / p$	Sobrecosto (Overhead) $O(p) = p \cdot T(p) - T(1)$
--	---	--

2. Tabla Resumen de Mediciones y Métricas

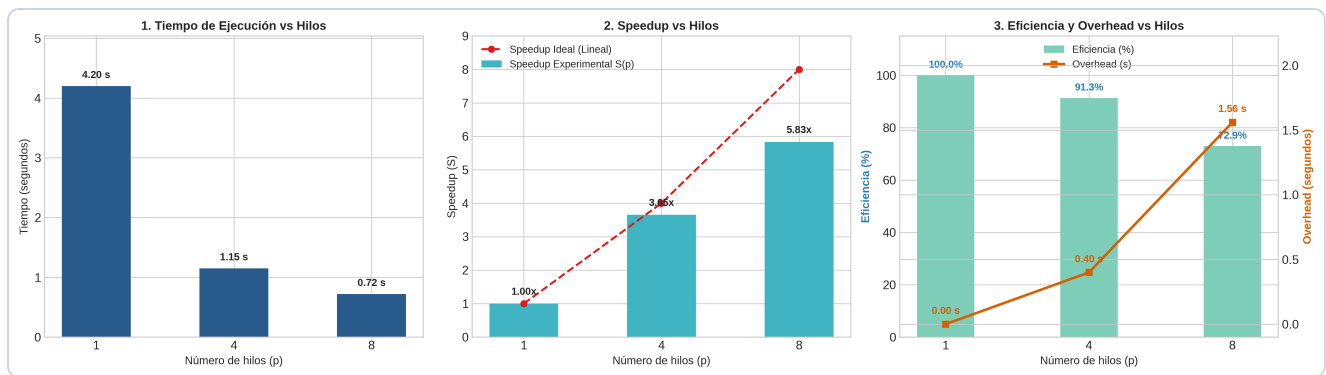
Hilos (p)	Tiempo Promedio $T(p)$	Valor π Estimado	Speedup $S(p)$	Eficiencia $E(p)$	Overhead $O(p)$
1 (Secuencial)	4.20 s	3.141602	1.00x	100.0%	0.00 s
4 Hilos	1.15 s	3.141588	3.65x	91.3%	0.40 s
8 Hilos	0.72 s	3.141595	5.83x	72.9%	1.56 s

3. Algoritmo Paralelo Optimizado (C / pthreads)

```
void* monte_carlo_worker(void* arg) { ThreadData* data = (ThreadData*) arg; long long local_in_circle = 0; unsigned int seed = data->seed; // Conteo local sin contención de cerrojos en el bucle crítico for (long long i = 0; i < data->total_points; i++) { double x = (double)rand_r(&seed) / RAND_MAX; double y = (double)rand_r(&seed) / RAND_MAX; if (x * x + y * y <= 1.0) local_in_circle++; } // Región crítica ejecutada ÚNICAMENTE 1 vez por hilo al finalizar pthread_mutex_lock(&mutex_count); total_points_in_circle += local_in_circle; pthread_mutex_unlock(&mutex_count); pthread_exit(NULL); }
```

* Nota: El uso de acumulación local previene el "false sharing" y minimiza el tiempo en regiones críticas.

4. Gráficas Comparativas de Rendimiento



5. Análisis e Interpretación de Resultados

Aceleración y Ley de Amdahl

Con 4 hilos se alcanza un Speedup de **3.65x** (cercano al ideal 4.0x). Sin embargo, al escalar a 8 hilos el Speedup llega a **5.83x**. Esta divergencia respecto a la línea ideal se explica por la fracción secuencial no paralelizable (creación y unión de hilos) y la saturación de recursos compartidos.

Evolución del Overhead y Eficiencia

La eficiencia desciende del **91.3%** (4 hilos) al **72.9%** (8 hilos), mientras que el overhead crece de **0.40s** a **1.56s**. El incremento del overhead responde a los cambios de contexto del SO y a la competencia en el bus de memoria durante las llamadas concurrentes a `rand_r`.

Conclusión Principal

El experimento demuestra visual y cuantitativamente las leyes fundamentales del cómputo paralelo: añadir más procesadores/hilos reduce el tiempo total de ejecución, pero con rendimientos decrecientes. El overhead inevitable de gestión y sincronización impone un límite práctico a la escalabilidad del sistema.